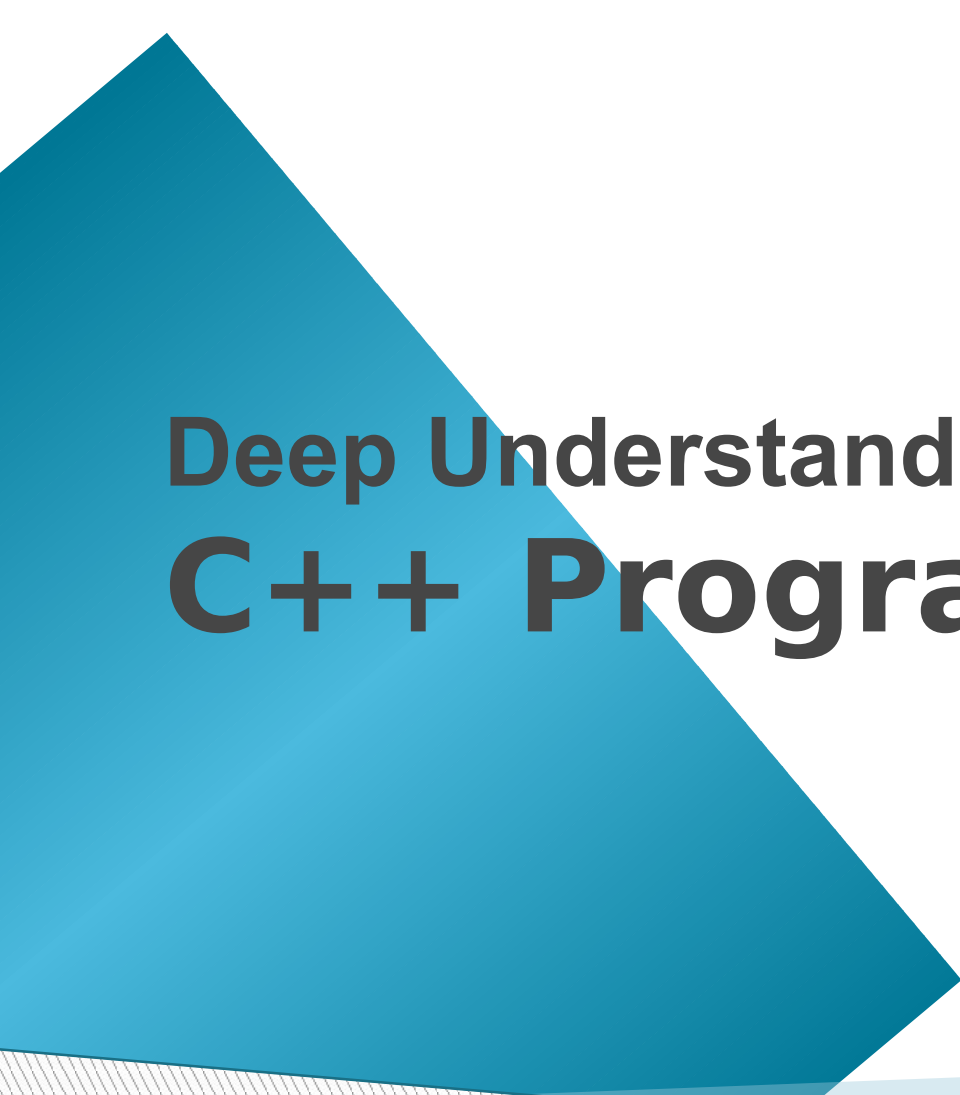
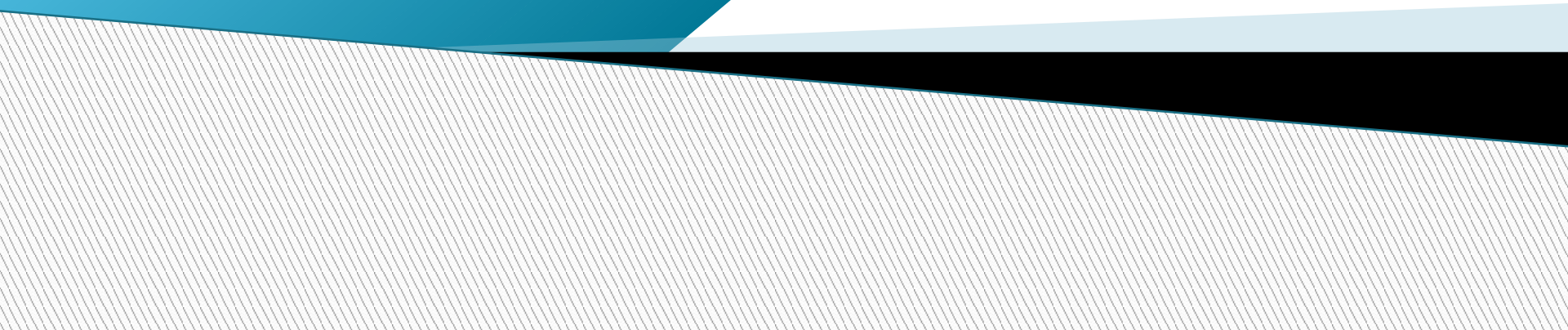


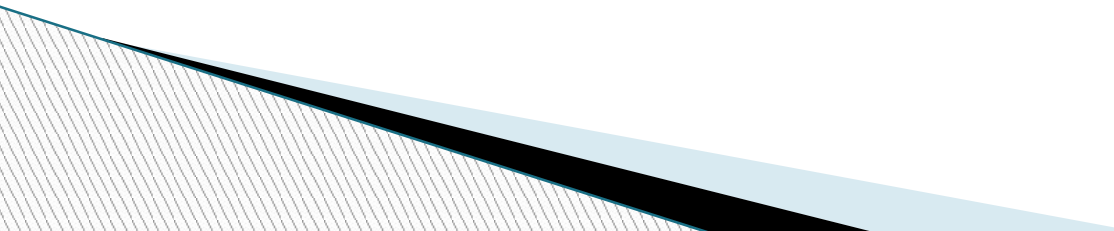
A large teal triangle pointing downwards, positioned on the left side of the slide, partially overlapping the title text.

Deep Understanding to **C++ Programming**

Dr.V.Arulkumar

A decorative graphic at the bottom of the slide consisting of a light blue wavy line, a black horizontal band, and a grey area with diagonal hatching below it.

Session Plan

- Programming Techniques
 - Introduction to Object Oriented Concepts
 - Basics of C++
 - Reference Variables
 - Parameter Passing techniques
 - Abstract Data types
 - Classes and Objects
 - Access Specifiers
 - Class – Best Practices and Guidelines
 - Constructors and Destructors
- 

Programming Techniques

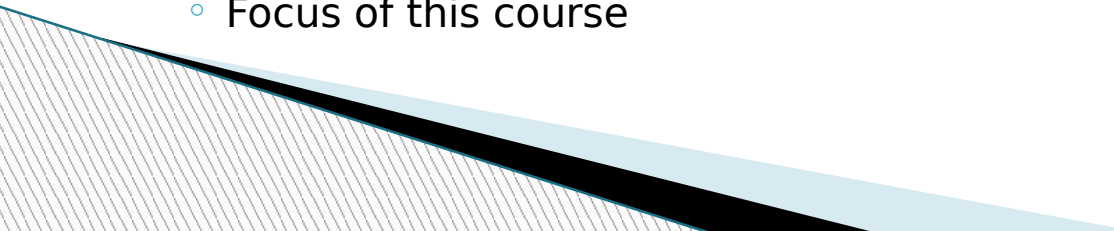
▶ **Unstructured**

- Sequence of instructions, which manipulates global data
- As size increases, code becomes more complex to maintain

▶ **Procedural Programming**

- Brought in Modularity in coding, enhancing maintainability
- Common functionalities grouped into separate modules
- Complexity made manageable by hiding complexity inside functions
Introduced the concept of structures (also known as data structures)

▶ **Object Oriented Programming**

- Data structures combined with relevant functions to create reusable objects
 - Focus of this course
- 

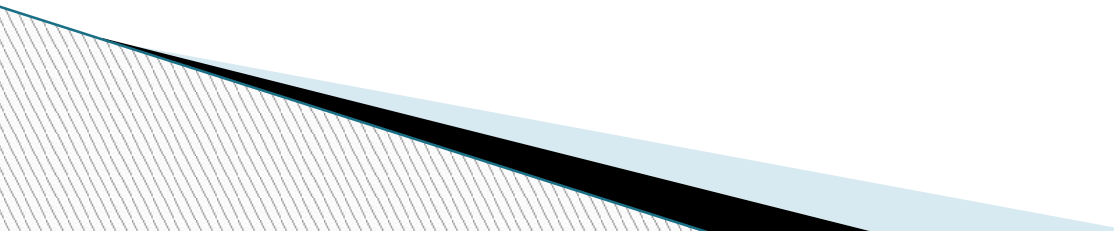
Procedural versus Object Oriented Programming

- In procedural programming, functions operate based on either local or global data
- In Object oriented programming, Objects exist and objects interact with other objects

▶ Click to edit Master text styles

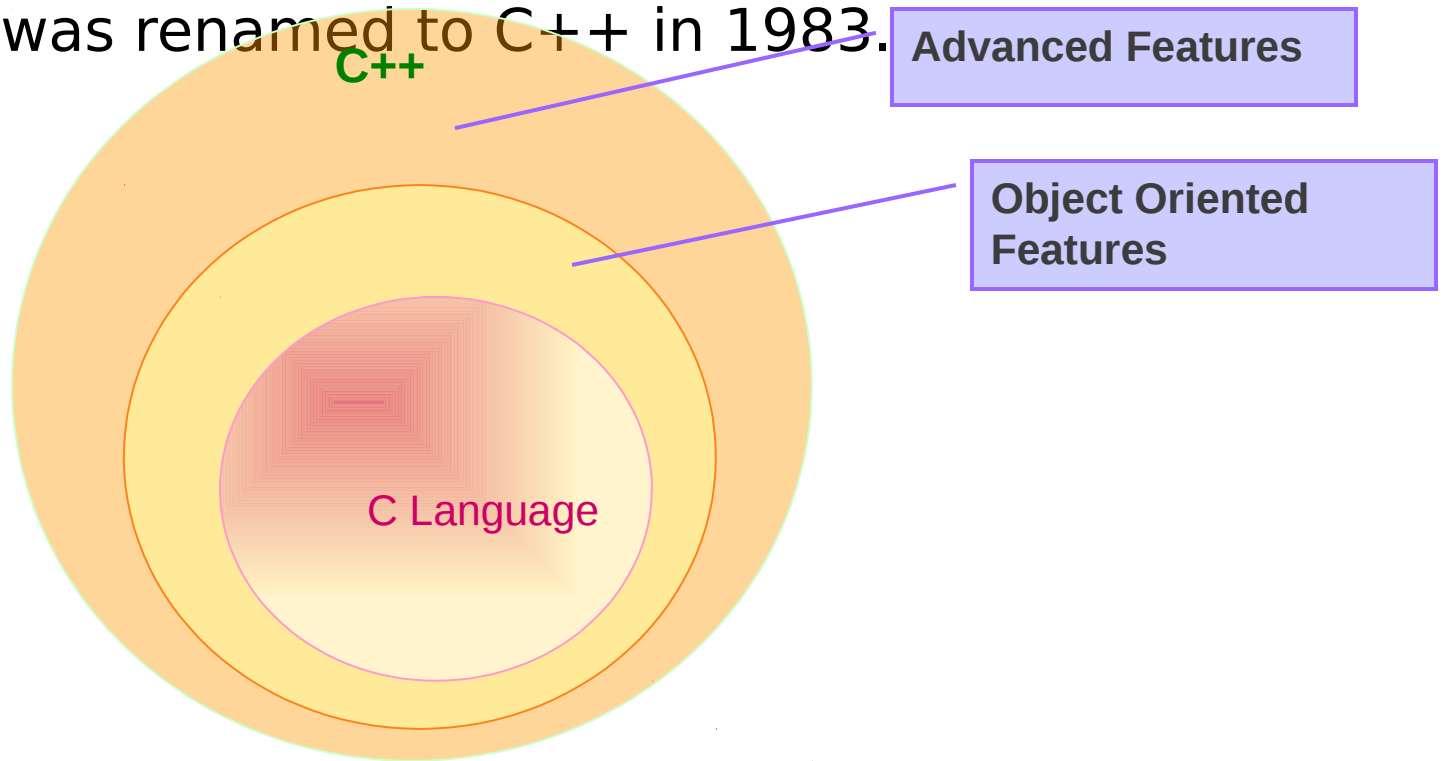
- Second level
- Third level
 - Fourth level
 - Fifth level

Object Oriented Concepts

- ▶ **Abstraction:** Process of forming of general and relevant information from a complex scenarios.
 - ▶ **Encapsulation:** Localization of information within an object. This also leads to Information Hiding
 - ▶ **Inheritance:** Is a process by which one object acquires the properties of another object
 - ▶ **Polymorphism:** allows the programmer to use “***one interface***” for “***many functions***”.
- 

Basics of C++

- ▶ C++ is an Object Oriented programming language
- ▶ It can be considered as a super set of C
- ▶ This enables backward compatibility
- ▶ **Bjarne Stroustrup** first invented “C with Classes” and then it was renamed to C++ in 1983.



Basics of C++ Comments

- ▶ C++ style comments are single line comments
- ▶ C++ also supports C Style comments (backward compatibility with C)

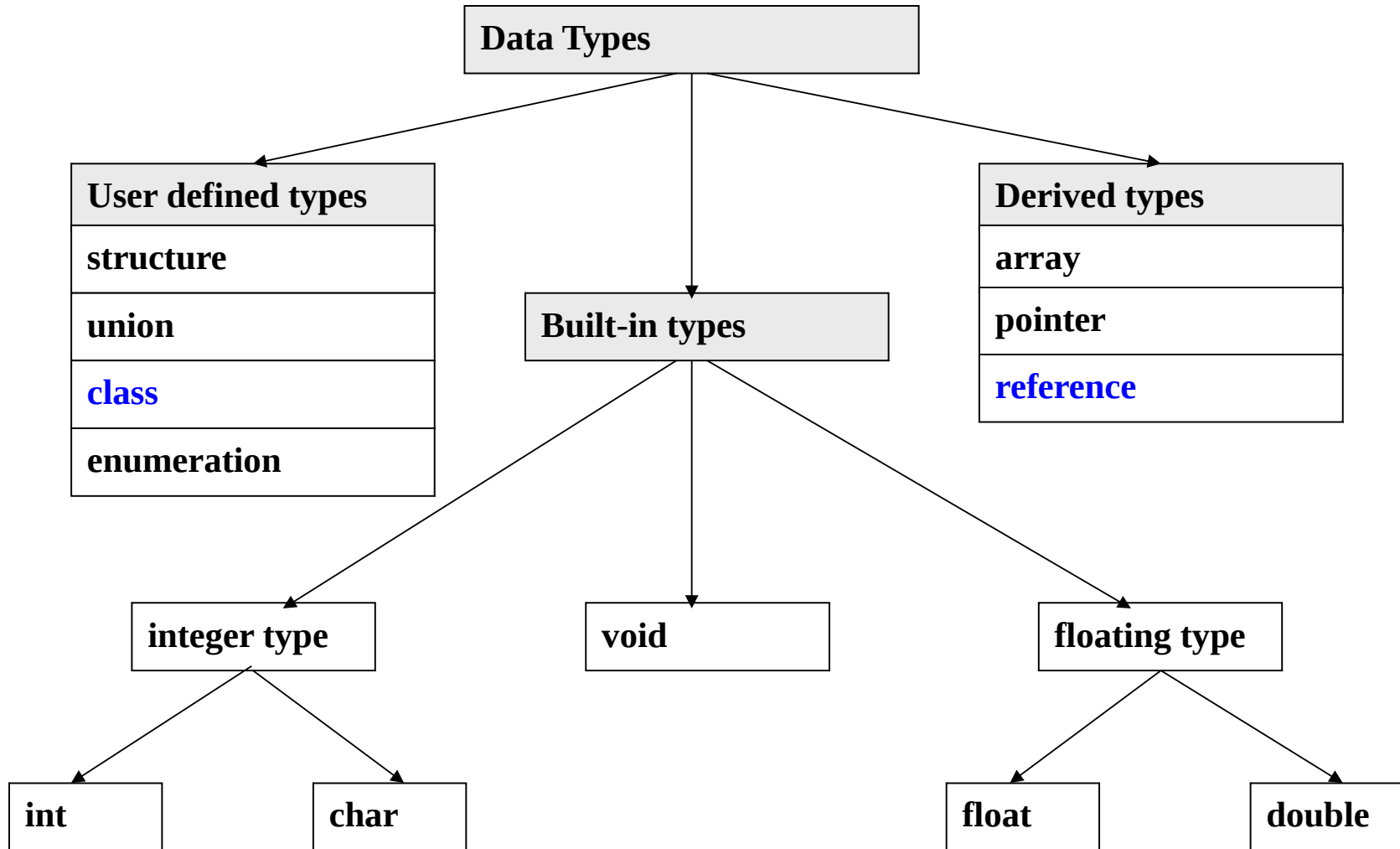
C- Style Comments:

```
/* C- Style Single Line Comment*/  
  
/  
*****  
***  
* This is C Style Multiple Line Comment  
* Used for Function Headers  
*****  
*** /
```

C++ Style Comments:

```
// This is a single line comment
```

Data types of C++



Variables and naming convention

- ▶ Variables can be declared anywhere in the program, just before it is used
- ▶ The naming convention followed for the variables need to be consistent and meaningful
- ▶ Hungarian Notation must be followed similar to C for variable naming conventions
- ▶ All primitive C Data types are supported in C++ due to backward compatibility

Prefix	Data Type	Example
i	int and unsigned int	iTotalMarks
f	float	fAverageMarks
d	double	dSalary
l	long and unsigned long	lFactorial
c	signed char and unsigned char	cChoice
ai	Array of integers	aiStudentId
af	Array of float	afquantity
ad	Array of double	adAmount
al	Array of long integers	alSample
ac	Array of characters	acEmpName

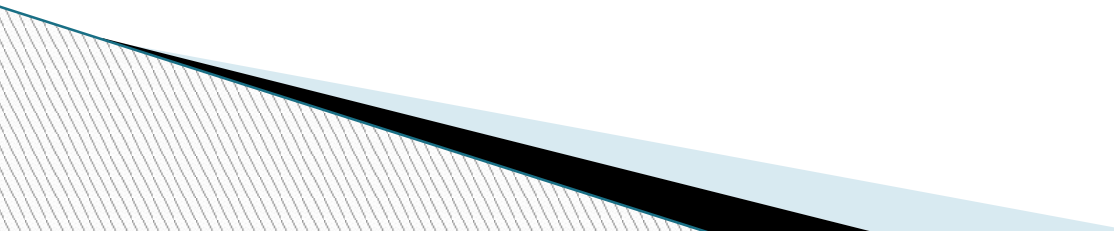
Control Structures

- ▶ Control Structures are statements which changes the execution sequence of the program
- ▶ C++ supports all the control structures supported in C
 - If, else if, else
 - switch case
 - for loop
 - while loop
 - do while

Operators Common in C and C++

- Arithmetic operators **+, -, *, /, %**
- Pre and Post Unary Operators **++, --**
- Assignment operator **=**
- Compound Assignment Operators **+=, -=, /=, *=, %=**
- Relational Operators **>, >=, <, <=, ==, !=**
- Logical Operators **!, &&, ||**
- Address Operator **&**
- **sizeof** operator
- Bitwise logical operators (**&** - AND, **|** - OR, **!** - XOR, **~** - NOT)
- Bitwise Shift operators (**<<** - Left Shift , **>>** - Right Shift)
- Conditional operator (**? :**) also called as Ternary Operator
 - Syntax:
 - **<var-name> = (<condition>) ? value-if-true : value-if-false ;**
 - **iMax=(iNum1>iNum2) ? iNum1 : iNum2 ;**

Operators in C++

- Scope Resolution Operator (`::`)
 - Memory Allocation/De-allocation operator (`new` and `delete`)
 - Member Access operators (`->` , `.*` , `->*` , `.`)
 - Reference Operator (`&`)
 - Insertion and Extraction Operators (`<<` and `>>`).
- 

Scope Resolution Operator (::)

- ▶ Used for resolving the scope of the variables
- ▶ Used for defining methods outside the class

```
# include <stdio.h>
int iNum=10;
int main(int argc,char** argv) {
    int iNum=20;
    if(20==iNum) {
        int iNum=30;
        printf("%d %d",iNum, ::iNum);
    }
    printf("\n%d %d",iNum, ::iNum);
    return 0;
}
```

30 10

20 10

Reference Variables

- ▶ Reference variable is an **alias** created for an already existing variable.
- ▶ Change in reference will change the value of the variable and vice versa
- ▶ Should be initialized when declared and cannot be reinitialized
- ▶ A variable can have multiple reference
- ▶ Array of reference is not allowed

```
int main(int argc,char** argv) {  
    int iNum=10;
```

```
        /* Declare and initialize reference variable */
```

```
    int& riNum = iNum;
```

```
    printf("%d %d",iNum,riNum);
```

```
        /* Change the value through reference variable */
```

```
    riNum=20;
```

```
    printf("\n%d %d",iNum,riNum);
```

```
    return 0;
```

```
}
```

10 10

20 20

Parameter Passing Techniques - Example

```
void SwapByAdd(int* piVal1,int* piVal2) {  
    int iTemp;  
    iTemp=*piVal1;  
    *piVal1=*piVal2;  
    *piVal2=iTemp;  
}
```

```
void SwapByRef(int& riVal1,int& riVal2){  
    int iTemp;  
    iTemp=riVal1;  
    riVal1=riVal2;  
    riVal2=iTemp;  
}
```

```
void SwapByValue(int iVal1,int iVal2){  
    int iTemp;  
    iTemp=iVal1;  
    iVal1=iVal2;  
    iVal2=iTemp;  
}
```

```
int main(int argc,char** argv)  
{  
    int iNum1=10,iNum2=20;
```

SwapByValue(iNum1,iNum2);

printf("%d %d",iNum1,iNum2);

SwapByAdd(&iNum1,&iNum2);

printf("%d %d",iNum1,iNum2);

SwapByRef(iNum1,iNum2);

printf("\n%d %d",iNum1,iNum2);

return 0;

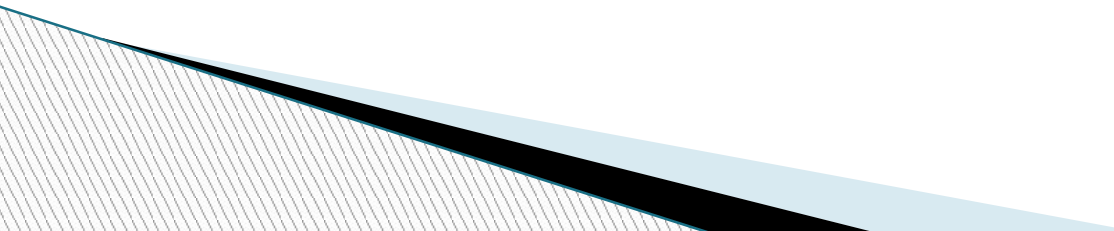
}

10 20

20 10

10 20

Functions - Pass By Pointer Versus Pass by Reference

- ▶ Referencing Offers a clean, elegant and efficient way to pass parameters to functions that changes the value of actual parameter
 - ▶ Difficulty in de-referencing is eliminated in pass by reference
 - ▶ Calling syntax is much simpler compared to call by address
 - ▶ Pass by Reference needs no memory
- 

Structures in C

- ▶ Composite data type
- ▶ Contains one or more logically related data (similar or dissimilar) grouped together under a single name for convenient handling
- ▶ Variables declared inside structure are called fields or members
- ▶ Functions cannot be part of Structure
- ▶ Structure variables can be created using structure name.
- ▶ Members can be accessed using <Structure variable Name>.<Member Name>

```
struct _employee {  
    int iEmpId;  
    float fBasic;  
    float fTotSal  
};
```

```
/* Structure variable creation */  
struct _employee E1;  
/* Accessing members */  
E1.iEmpId=43002  
E1.fBasic=2500.00  
E1.fTotSal=E1.fBasic+300;
```

Structures in C++

- ▶ C++ **extends the features** of C Structures
- ▶ C++ structures can have functions as its members.
- ▶ Default access specifier for Structure members is “public”. Access specifiers will be dealt in later slides.

```
struct _employee {  
    int m_iEmpId;  
    float m_fBasic;  
    float m_fSal;  
    void CalculateSal() {  
        /* Code for sal  
        calculation */  
    }  
}
```

```
struct <structure_name> {  
    private:  
        /* private data member and  
        methods */  
    protected:  
        /* protected data member and  
        methods */  
    public:  
        /* public data member and  
        methods */  
};
```

Class

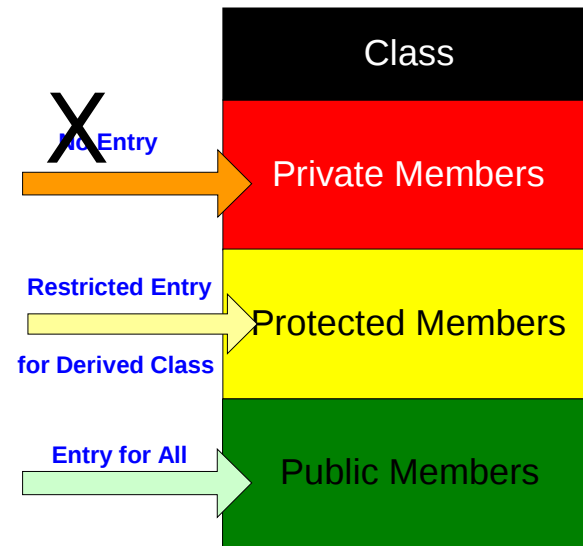
- ▶ Basic unit of abstraction in C++
- ▶ Extension of the concept of structures
- ▶ Provides a technique to bind data and functions together (Encapsulation)
- ▶ Data Members – Variables declared inside the class
- ▶ Member Functions – Functions declared/defined inside the class
- ▶ Member Functions are also called as Methods

```
class Employee {  
    int m_iEmpId;  
    float m_fBasic;  
  
public:  
    void CalculateSal() {  
        /* Code for sal  
        calculation */  
    }  
}
```

```
class <class_name> {  
private:  
    /* private data member and  
    methods */  
protected:  
    /* protected data member and  
    methods */  
public:  
    /* public data member and  
    methods */  
};
```

Class - Implementation of Encapsulation (Access Specifiers)

- Public Members
 - Accessed within and outside the class
- Protected Members
 - Accessed within and only by derived classes
- Private Members
 - Accessed only within the class
- Good design practice is to keep data members under private and methods under public
- Methods invoked ONLY by other methods of the same class are kept private
- Such methods are called **helper** methods



Objects

- ▶ Instance of a class in memory
- ▶ Unique, identifiable, self – contained entity that contains attributes and behaviors
- ▶ If class is viewed as a data type then object can be considered as a variable of class data type.
- ▶ **Ex:** 'Arun' is an Object of Trainee Class

- ▶ Syntax for creating objects
 <*Class_Name*> *Object1, Object2,.....;*
- ▶ **Ex: Trainee oT1, oT2;**

- ▶ Syntax for Invoking Methods
 - *Object1.Method1();*
 - *Object1.Method2();*
- ▶ **Ex: oT1.TakeTest();
 oT2.GetResults();**

Class and Objects - Example

```
class Trainee {  
private:  
    int m_iEmpId;  
    char m_acEmpName[25];  
    float m_fBasic;  
    float m_fHRA;  
public:  
    void SetData(int iEmpId,  
        char acEmpName[],float  
        Basic,  
        float fHRA) {  
        ..... }  
    void CalculateSal() {  
        ..... }  
    void CalculateTax() {  
        .....}  
}; //class Trainee ends here
```

```
int main(int argc,char** argv) {  
    /* Object Creation */  
    Trainee oT1;  
  
    /* Invoking SetData */  
  
    oT1.SetData(101,"Henry",1200,1  
50)  
  
    /* Invoking CalculateSal */  
    oT1.CalculateSal();  
  
    /* Invoking CalculateTax */  
    oT1.CalculateTax();  
  
}
```

Class Access Specifiers - Example

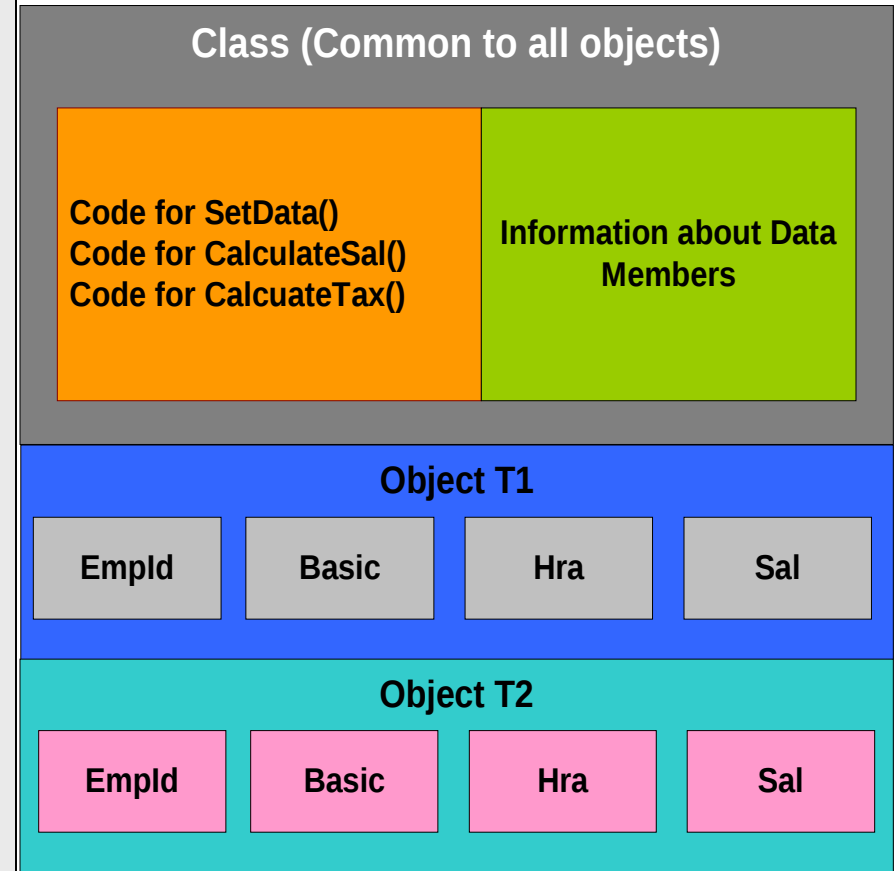
```
class Trainee {  
    int m_iEmpId;  
    char m_acEmpName[30];  
    float m_fBasic;  
    float m_fHRA;  
  
    public:  
        void SetData(int iEmpId,  
            char acEmpName[],float  
fBasic,  
            float fHRA) {  
            .....  
        }  
        void CalculateSal() {  
            .....  
        }  
        void CalculateTax() {  
            .....  
        }  
};
```

```
int main(int argc,char** argv) {  
    /* Object Creation */  
    Trainee oT1;  
    /* Invalid - Accessing  
private  
members outside the class */  
    oT1.m_iEmpId=43003;  
    oT1.m_fBasic = 1200.00  
    oT1.m_fHRA=350.00  
  
    /* Valid - Accessing public  
members  
outside the class */  
  
    oT1.SetData(101,"Dave",12  
    00,350)  
    oT1.CalculateSal();  
    oT1.CalculateTax();  
}
```

Memory allocation for Classes and Objects

```
class Trainee {  
private:  
    int m_iEmpId;  
    float m_fBasic;  
    float m_fHRA;  
    float m_fSalary;  
public:  
    void SetData(int iEmpId,  
float fBasic,  
        float fHRA){  
    }  
    void CalculateSal() {  
        // code goes here  
    }  
    void CalculateTax() {  
        //code goes here  
    }  
}
```

```
Trainee oT1,oT2;
```



Class - Defining methods outside the class

/* Class Declaration */

```
class Trainee {  
private:  
    int m_iEmpId;  
    char m_acEmpName[25];  
    float m_fBasic;  
    float m_fHRA;  
public:  
    void SetData(int iEmpId,  
                  char cEmpName[],  
                  float fBasic,float fHRA) ;  
    void CalculateSal() ;  
    void CalculateTax();  
};
```

/* Method Definitions */

```
void  
    Trainee::SetData(iEmpID,acEmpNa  
me,  
                                fBasic,fHra){  
        ... ../* Code to set the data */  
}  
  
void Trainee::CalculateSal() {  
    ..... /* Code to calculate  
salary */  
}  
  
void Trainee::CalculateTax() {  
    /* Code to calculate Tax */
```

/* Object Creation */

```
Trainee oT1;  
oT1.SetData(43003,"Dave",1200,350);
```

Class - Coding Standards, Best Practices and Guidelines

► Coding Standards

- Member variables - m_<Variable Name>
- m_iEmpld, m_fBasic, m_acName
- Methods - Hungarian Notation
- SetEmployeeNumber(), CalculateSalary(), CalculateTax()

◦ Object - o<Object_Name>

 oTrainee1, oTrainee2
Employee.h Employee.cpp

 mployee_main.cpp

Constructors

- ▶ Used for automatic initialization of instance variables during Object creation.
- ▶ Has the same name of the class and do not have return types (not even void)
- ▶ Invoked automatically when an object of its associated class is created

```
/* Class Declaration */  
class Trainee {  
private:  
    int m_iEmpId;  
    char *m_pcName;  
public:  
    /* Declaration of  
    Constructor */  
    Trainee();  
    /* Declaration of other  
    methods */  
    void CalculateSal() ;  
    void CalculateTax();  
};
```

```
/* Constructor Definition */  
Trainee::Trainee() {  
    /* Initial Values */  
    m_iEmpId=-1;  
    m_pcName=NULL;  
}  
/* Definition of other methods */  
void Trainee::CalculateSal() {  
    ..... /* Code to calculate  
    salary */  
}  
void Trainee::CalculateTax() {  
    ..... /* Code to calculate Tax  
    */  
}
```

Constructors - Types

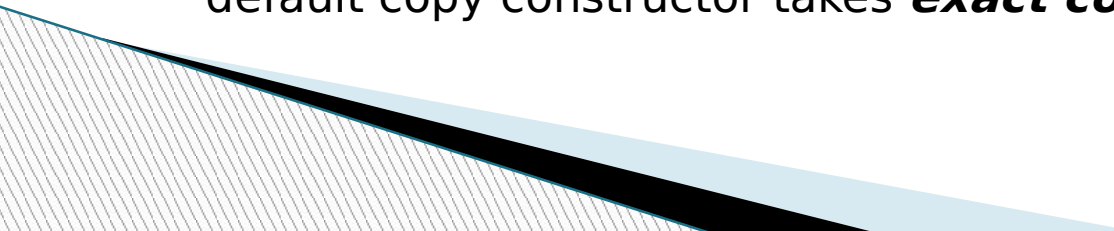
▶ Default Constructor

- Takes no parameters and no return value (**NOT even void**)
- Automatically written by C++ if no user-defined constructor is present
- The programmer can **redefine** the default constructor.

▶ Parameterized Constructor

- Takes one or more parameters
- These arguments can be used for initializing the objects when created

▶ Copy Constructor

- Takes only one argument which is of same class type
 - Generally used to copy **some selective** data members of one object to another
 - Automatically written by the compiler if no user defined copy constructor is present.
 - default copy constructor takes **exact copy** of one object to another
- 

/* Class Declaration */

```
class Trainee {  
private:  
    int m_iEmpId;  
    char *m_pcName;  
public:  
    Trainee();  
    Trainee(int Id, char  
Name[25]);  
    Trainee(Trainee&  
RefTrainee);  
    void CalculateSal();  
};
```

```
Trainee oT1;  
Trainee oT2(43003,"David John");  
Trainee oT3=oT1;
```

/* Default Constructor */

```
Trainee::Trainee() {  
    m_iEmpId=0;  
    m_pcName=NULL;  
}
```

/* Parameterized Constructor */

```
Trainee::Trainee(int Id, char  
Name[25]){  
    m_pcName=malloc(strlen(Name)+1);  
    strcpy(m_pcName,Name);  
    m_iEmpId=Id;  
}
```

/* Copy Constructor */

```
Trainee::Trainee(Trainee& rt) {  
    int len=strlen(rt.m_pcName)+1;  
    m_pcName=malloc(len+1);  
    strcpy(m_pcName,rt.m_pcName);  
    m_iEmpId=rt.m_iEmpId;  
}
```

Constructors - More about Copy Constructors

- A **copy** constructor is one which takes only one argument of the same class type.

```
/* Copy Constructor */
```

```
Trainee::Trainee(Trainee & rt) {  
    int len=strlen(rt.m_pcName)+1;  
    m_pcName=malloc(len+1);  
    strcpy(m_pcName,rt.m_pcName);  
    m_iEmpId=rt.m_iEmpId;  
}
```

Here *rt* is a reference object.

Generally, the copy constructor is used when you want to copy a part of data members of one object to a new object.

```
Trainee oT1, oT2;  
oT2 = oT1;    // takes exact  
copy
```

// or bit-by-bit copy

- **The copy constructor is called in 4 different situations:**

- `Trainee oT1(oT2);` // Copy constructor is called
- `Trainee oT3 = oT2;` // Copy constructor is called
- `oT3 = oT2;` // Here a bit by bit copy operation takes place
- `void Display(oT1, oT2);` // When the object is passed as parameter. And
- `oT2 = Compare(oT3, oT4);` // When the object is returned to calling point.

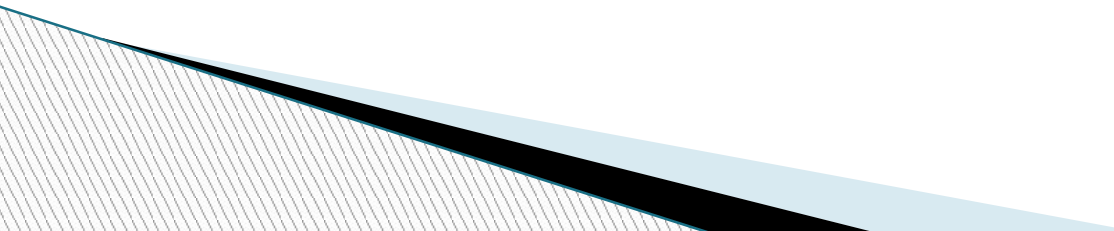
Destructor

- is a method which automatically gets invoked when an object is destroyed
- has the same name as the class and preceded by a tilde(~)
- takes no arguments & No return value. So, a class can have only one destructor.
- used for housekeeping jobs like releasing memory, resources...etc

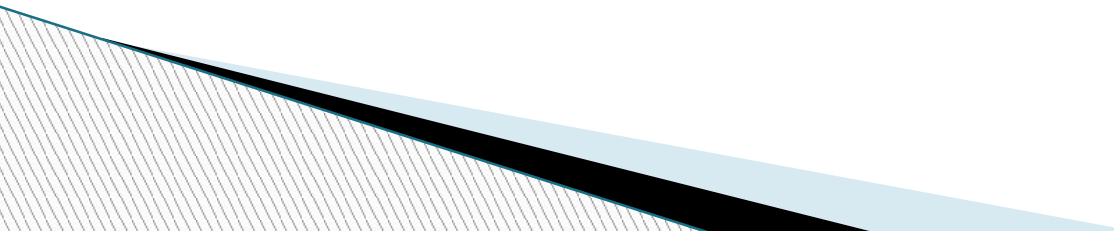
```
/* Class Declaration */  
class Trainee {  
private:  
    int m_iEmpId;  
    char *m_pcName;  
public:  
    Trainee();  
    Trainee (int Id, char  
    Name[25]);  
    ~Trainee();  
};
```

```
Trainee::Trainee(int Id,char Name[]) {  
  
    m_pcName=malloc(strlen(Name)  
    +1);  
    ...../ * Code */  
}  
  
Trainee::~~Trainee() {  
    if(m_pcName!=Null) {  
        free(m_pcName);  
    }  
  
}
```

Summary

- ▶ Programming Techniques
 - ▶ Introduction to Object Oriented Concepts
 - ▶ Basics of C++
 - ▶ Reference Variables
 - ▶ Parameter Passing techniques
 - ▶ Abstract Data types
 - ▶ Classes and Objects
 - ▶ Access Specifiers
 - ▶ Class – Best Practices and Guidelines
 - ▶ Constructors and Destructors
- 

Session Plan 2

- ▶ Dynamic Memory Allocation
 - ▶ Inline Functions
 - ▶ Static Members
 - ▶ Const Members
 - ▶ Polymorphism
 - ▶ 'this' pointer
 - ▶ Aggregation
 - ▶ Cin & Cout
- 

Pointer to objects

```
class Trainee {
private:
    int m_iEmpId;
    char m_acName[30];
    float m_fBasic;
    float m_fHRA;
public:
    void
    SetData(ID,Name,Basic,Hra){
        .....//Code
    }
    void CalculateSal() {
        .....//Code
    }
    void CalculateTax(){
        .....//Code
    }
};
```

```
int main(int argc,char** argv) {
    Trainee oT1;

    //Declaring a pointer to an
    Object
    Trainee* poT1;

    //Initializing pointer
    poT1=&oT1;

    //Invoking Methods using
    pointer
    poT1-
    >SetData(102,"Dave",1200,350);
    poT1->CalcuatSal();
    poT1->CalculateTax();

    return 0;
}
```

Dynamic Memory Allocation - new & delete operators

- ▶ The **new** and **delete** operators are used to allocate and free memory at run time.
- ▶ The **new** operator allocates memory and returns a pointer to the start of it.
- ▶ The **delete** operator frees memory previously allocated using **new**.
- ▶ The general forms of **new** and **delete** are shown here:
 - p_var = **new** type;
 - **delete** p_var;
- ▶ Here, p_var is a pointer variable that receives a pointer to memory that is large enough to hold an item of type **type**.
- ▶ ***Trainee *poT1 = new Trainee();***
vs Trainee *poT1 = (Trainee *) malloc(sizeof(Trainee));
- ▶ Even though malloc/free allocates and releases the memory respectively, Constructor/Destructor is not called during dynamic object creation/destruction.
- ▶ Hence use new/delete for dynamic memory allocation for objects and malloc/free for primitive data types.

Dynamic Memory Allocation - Example using new & delete

```
class Trainee {  
private:  
    int m_iEmpId;  
    char m_acName[30];  
    float m_fBasic;  
    float m_fHRA;  
public:  
    void  
        SetData(ID,Name,Basic,Hra  
        ){  
            .....//Code  
        }  
    void CalculateSal() {  
        .....//Code  
    }  
    void CalculateTax(){  
        .....//Code  
    }  
};
```

```
int main(int argc,char** argv) {  
    //Declaring a pointer to an  
    Object  
    Trainee* poT1;  
    //Initializing pointer  
    poT1=new Trainee();  
    //Invoking Methods using  
    pointer  
    poT1-  
    >SetData(102,"Dave",1200,350);  
    // Releasing Memory  
    delete poT1;  
  
    poT1=new  
    Trainee(101,"Dave",1200,350);  
    poT1->CalculateSal();  
    delete poT1;
```

Memory allocation through new and delete

- ▶ The dynamic memory allocation happens on the heap.
- ▶ Heap is a memory space which grows or shrinks dynamically.
- ▶ Whenever the memory is allocated dynamically for any object, a space is allocated in the heap. This allocation remains until it is deleted (freed).
- ▶ If a program fails to de-allocate memory, that memory cannot be used by other programs. This is called as “**memory leak**”.

```
// Example of Memory leak
int main(int argc, char** argv) {
    Trainee* poT1 = new Trainee();
    poT1->SetData(102, "Dave", 1200, 350);
    poT1->CalculateSal();
    return 0;
}
```

Inline Functions

- Normal function-calls eat up lot of CPU time for its bookkeeping activities
- So, for smaller methods, the C++ compiler automatically replaces the function calls with the actual code to be executed.
- As a programmer, we can also ***request*** the compiler to do so.
- This is done usually for smaller and non-recursive functions.
- Inline is a suggestion given to the compiler. Compiler decides whether to replace or not.

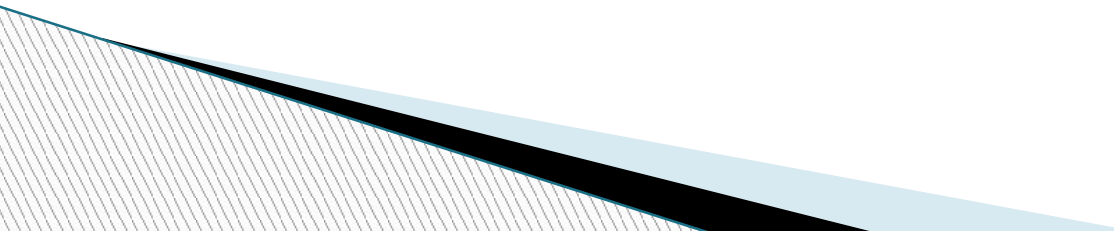
```
inline double CalcArea() {  
    return Length * Breadth;  
}
```

Static Members

Static Data Members

- Data member whose value is shared by all the objects of the class
- Static data is allocated memory only once when the class is loaded.
- Must be defined outside the class
- Also can be initialized during definition, If not initial value will be 0.

Static Member Functions

- Provide a very generic functionality that doesn't require us to instantiate the class.
 - can be invoked without any objects (ie. using class name)
 - can access only other static members (static functions or static variables).
- 

```

class Trainee {
private:
    int m_iEmpId;
    static int
m_iTotalTrainees;
public:
    Trainee(int iEmpID);
    static int
GetTotalTrainees();
    ~Trainee();
};

```

```

Trainee::Trainee(int iEmpId)
    m_iTotalTrainees++;
    m_iEmpid=iEmpId;
}
int Trainee::GetTotalTrainees()
    return (m_iTotalTrainees);
}
Trainee::~~Trainee() {
    m_iTotalTrainees--;
}
Trainee::m_iTotalTrainees=0;
MUST

```

```

// Main Program
Trainee *poT1,*poT2;
printf("%d",Trainee::GetTotalTrainees());
poT1= new Trainee(1001);
printf("%d",Trainee::GetTotalTrainees());
poT2= new Trainee(1002);

```

```

// Main Program continues
printf("%d",Trainee::GetTotalTrainees());
delete poT1;
printf("%d",Trainee::GetTotalTrainees());
delete poT2;
printf("%d",Trainee::GetTotalTrainees());

```

0

1

2

1

0

'const' Member Functions

- ▶ We can prevent methods from modifying data members
- ▶ Guarantees that value of data member will not get modified inside constant methods
- ▶ Member functions like getter methods, display methods can be made const

```
class Trainee {  
private:  
    int m_iEmpId;  
    float m_fSal;  
public:  
    int GetEmpId() const;  
    float GetSal() const;  
    void SetSal(float fSal);  
    void SetEmpId(int  
iEmpId);  
};
```

```
void Trainee::SetEmpId(int iEmpID)  
{  
    m_iEmpId= iEmpID;  
}  
void Trainee::SetSal(float fSal) {  
    m_fSal=fSal;  
}  
int Trainee::GetEmpId() const{  
    return(m_iEmpId);  
}  
float Trainee::GetSal() const{  
    m_fSal=m_fSal+1000;  
    return(m_fSal);  
}
```

Invalid

const Objects

- Objects which cannot modify the value of data members
- It can invoke only static & const methods

```
class Trainee {  
private:  
    int m_iEmpId;  
    float m_fSal;  
public:  
    int GetEmpId() const;  
    void SetEmpId(int  
        iEmpId);  
};
```

```
void Trainee::SetEmpId(int iEmpId)  
{  
    m_iEmpId=iEmpId;  
}  
int Trainee::GetEmpId() const{  
    return(m_iEmpId);  
}
```

Invalid

```
const Trainee oT1;  
oT1.SetEmpId(100);  
printf("%d",oT1.GetEmpId());
```

Function - Default Arguments

- ▶ Allows function call without supplying all arguments
- ▶ Function assigns the default value to the argument which is during function invocation
- ▶ Should be on the trailing end
- ▶ Should be specified either in the function declaration or definition

SetData(1001,1200
)

SetData(1001,1000
)

Invalid

```
class Trainee {  
private:  
    int m_iEmpId;  
    float m_fSal;  
public:  
    void SetData(int iEmpId,float  
        fSal=1000);  
    void CalculateSal();  
};
```

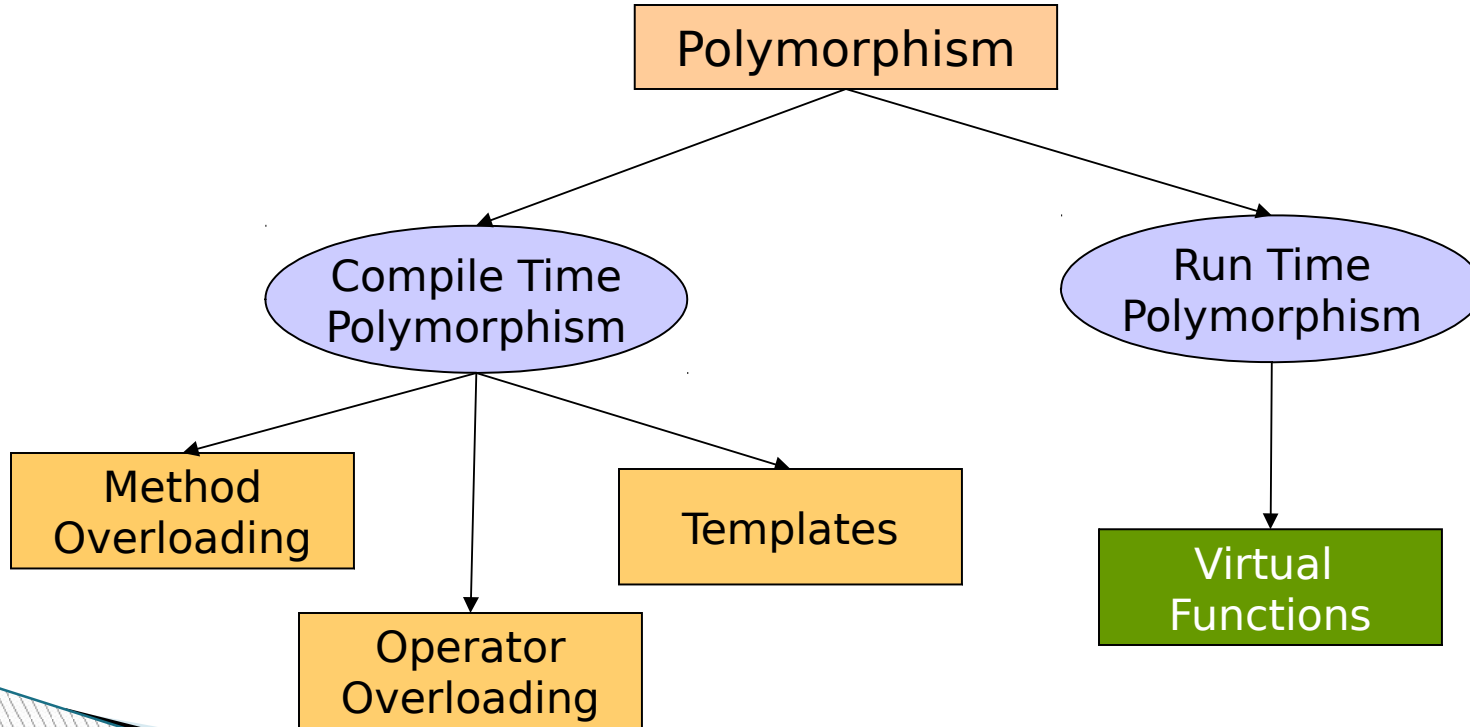
```
void Trainee::SetData(int  
    iEmpId, float fSal) {  
    m_iEmpId=iEmpId;  
    m_fSal=fSal;  
}
```

```
void Trainee::CalculateSal() {  
    // Code to calculate Sal  
}
```

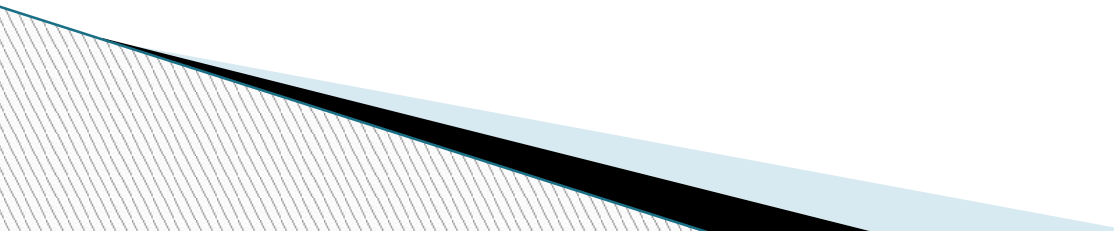
```
Trainee oT1;  
oT1.SetData(1001,1200);  
oT1.SetData(1001);  
oT1.SetData();
```

Polymorphism

- ▶ Is a Greek word, which means one name, multiple forms.
- ▶ Is the ability of objects to respond differently to the same message. The kind of response is dependent on the data types used at a given instance.
- ▶ Allows objects having different internal structure to share the same external interface.



Polymorphism - Binding

- ▶ Binding refers to the process of linking a procedure call to the code to be executed in response of the call.
 - ▶ Are of two types.
 - Static binding or Early or Compile time binding and
 - Dynamic binding or Late or Runtime binding.
 - ▶ In **Static binding** the code associated with a given procedure call is resolved during compiler time.
 - ▶ In **Dynamic binding** the code associated with a given procedure call is resolved during run-time.
- 

Function Overloading

- ▶ In C++, we can have more than one function with the same name.
- ▶ The signature of each overloaded function must be unique.
- ▶ The signature refers to the **number, type and order of the parameters**.
- ▶ The compiler will determine which version of the function is being invoked by analyzing the parameters.

```
int abs(int iNum) {  
    if(iNum<0) iNum=-iNum;  
    return (iNum);  
}  
float abs(float fNum) {  
    if(fNum<0) fNum=-fNum;  
    return (iNum);  
}
```

```
int main(int argc, char * argv[] ) {  
    int iNum = -10;  
    float fNum = 2.3f;  
    printf("%d",abs(iNum));  
    printf("%f",abs(fNum));  
}
```

Function Overloading and Ambiguity

```
void fnOne(int iValue1,int iValue2=10) {  
    printf("Inside function with two parameters:%d  
    %d",iValue1,iValue2);  
}  
  
void fnOne(int iValue1) {  
    printf("Inside function with one parameters: %d",iValue1);  
}  
  
int main(int argc,char** argv) {  
    fnOne(21);  
    /*compiler won't be able to decide which function to be  
    invoked*/  
  
    fnOne(10,20);//no ambiguity  
}
```

Constructor overloading

- Similar to functions, methods can also be overloaded
- Class can have more than one methods with same name and different implementations
- Constructors can also overloaded
- Destructors ***CANNOT BE*** overloaded

Method Returning Object - Example

```
class Circle {  
private:  
    int m_iRadius;  
public:  
    Circle(int iRadius);  
    float Area();  
    Circle ScaleUp(int  
iTimes);  
};
```

```
Circle::Circle(int iRadius){  
    m_iRadius=iRadius;  
}  
float Circle::Area() {  
  
    return(m_iRadius*m_iRadius*3.1  
4);  
}  
Circle ScaleUp(int iTimes) {  
    Circle  
    oTemp(m_iRadius*iTimes);
```

```
//Usage  
Circle oC1(1);  
printf("%f",oC1.Area());  
Circle oC2=oC1.ScaleUp(2);  
printf("%f %f",oC1.Area(), oC2.Area());
```

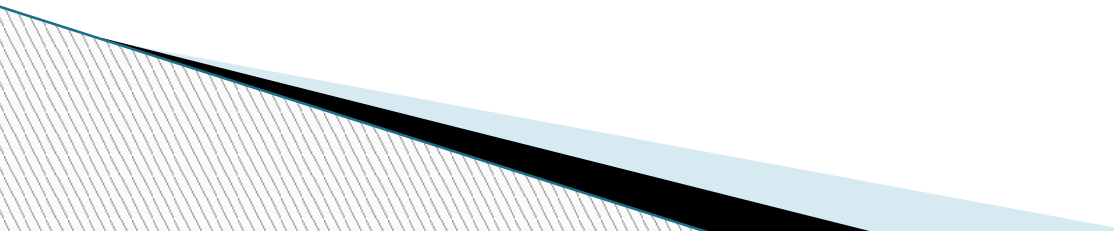
Cascading Method Call - Example

```
class Circle {  
Private:  
    int m_iRadius;  
public:  
    Circle(int iRadius);  
    float Area();  
    Circle ScaleUp(int iTimes);  
};
```

```
Circle::Circle(int iRadius){  
    m_iRadius=iRadius;  
}  
float Circle::Area() {  
    return(m_iRadius*m_iRadius*3.14);  
}  
Circle ScaleUp(int iTimes) {  
    Circle  
    oTemp(m_iRadius*iTimes);
```

```
//Main Program  
Circle oC1(1);  
printf("%f",oC1.Area());  
float fArea = oC1.ScaleUp(2).Area();  
printf("%f",fArea);
```

'this' pointer

- ▶ 'this' pointer is a pointer that is automatically available in all non-static member functions of a class.
 - ▶ Allows objects to access its own address within member functions
 - ▶ Points to the current object, that is, object that invoked the method.
 - ▶ When a member function is called, it is automatically passed an implicit argument that is a pointer to the invoking object
 - ▶ Since static functions can be called even without objects, 'this' pointer cannot be used with static member functions.
- 

'this' pointer - Example

```
class Circle {  
private:  
    int iRadius;    // no m_  
public:             // this is  
intentional  
    void SetRadius(int  
iRadius);  
    void GetRadius()  
    float Area();  
};
```

```
void Circle::SetRadius(int iRadius) {  
    this->iRadius=iRadius;  
}  
int Circle::GetRadius() {  
    return(iRadius);    //return(this-  
>iRadius)  
}  
float Circle::Area() {  
    return(3.14f*iRadius*iRadius);  
}
```

//Main Program

```
Circle oC1(1);  
printf("%f",oC1.Area());
```

Method Returning Object Using 'this' pointer - Example

```
class Circle {  
private:  
    int m_iRadius;  
public:  
    Circle(int iRadius);  
    float Area();  
    Circle ScaleUp(int iTimes);  
};
```

```
Circle::Circle(int iRadius){  
    m_iRadius=iRadius;  
}  
float Circle::Area() {  
    return(m_iRadius*m_iRadius*3.14)  
}  
Circle ScaleUp(int iTimes) {  
    this->m_iRadius=m_iRadius*iTimes;
```

```
//Main Program
```

```
Circle oC1(1);  
printf("%f",oC1.Area());  
Circle oC2=oC1.ScaleUp(2);  
printf("%f %f",oC1.Area(), oC2.Area());
```

Method Returning Reference to an Object - Example

```
class Circle {  
private:  
    int m_iRadius;  
public:  
    Circle(int iRadius);  
    float Area();  
    Circle & ScaleUp(int iTimes);  
};
```

```
Circle::Circle(int iRadius){  
    m_iRadius=iRadius;  
}  
Float Circle::Area() {  
    return(m_iRadius*m_iRadius*3.14)  
}  
Circle& ScaleUp(int iTimes) {  
    this->m_iRadius=m_iRadius*iTimes;  
}
```

```
//Main Program  
Circle oC1(1);  
printf("%f",oC1.Area());  
Circle oC2=oC1.ScaleUp(2);  
printf("%f %f",oC1.Area(), oC2.Area());
```

Method Returning pointer to an Object - Example

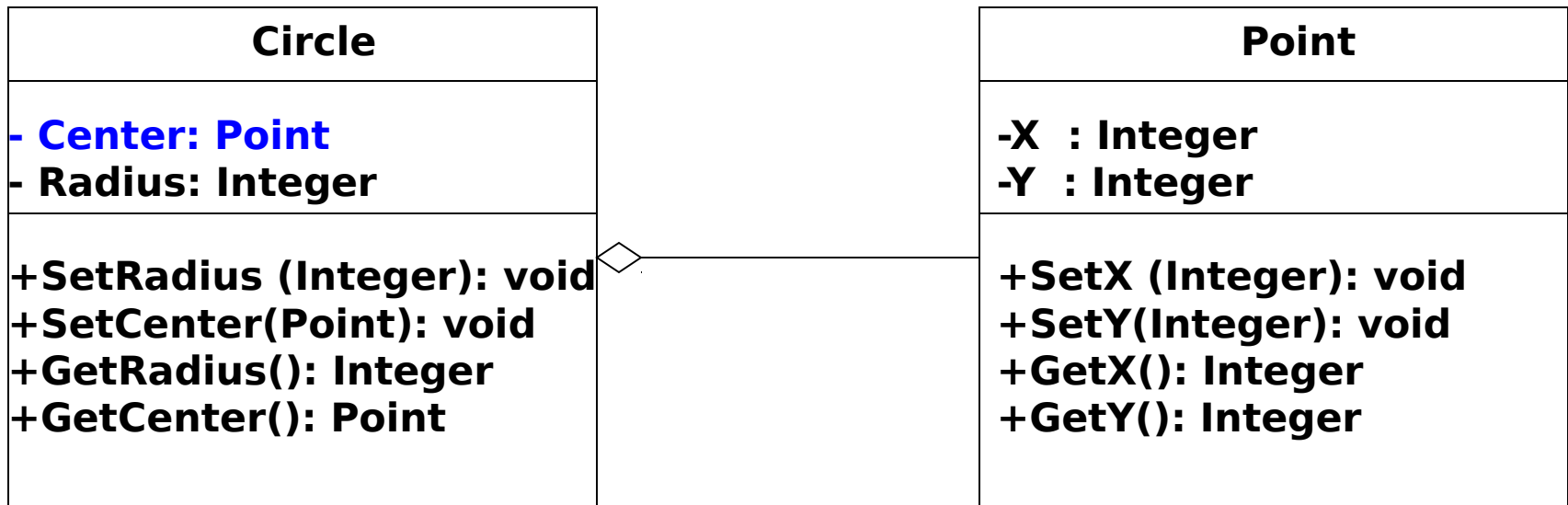
```
class Circle {  
Private:  
    int m_iRadius;  
public:  
    Circle(int iRadius);  
    float Area();  
    Circle* ScaleUp(int iTimes);  
};
```

```
Circle::Circle(int iRadius){  
    m_iRadius=iRadius;  
}  
float Circle::Area() {  
    return(m_iRadius*m_iRadius*3.14f);  
}  
Circle* ScaleUp(int iTimes) {  
    this->m_iRadius=m_iRadius*iTimes;
```

```
//Main Program  
Circle oC1(1);  
Circle* poC1;  
printf("%f",oC1.Area());  
poC2=oC1.ScaleUp(2);  
printf("%f %f",oC1.Area(), poC2->Area());
```

Aggregation

- ▶ refers to a relationship where one class contains objects of another class as it's data members
- ▶ is also called as “**Has-A**” relationship and also called as “**Containership**”



Aggregation - Example (1 of 3)

// Class Point Declaration

```
class Point {  
private:  
    int m_iX;  
    int m_iY;  
public:  
    Point(int iX, int iY);  
    void SetX(int iX);  
    void SetY(int iY);  
    int GetX();  
    int GetY();  
};
```

// Class Point Implementation

```
Point::Point(int iX,int iY) {  
    m_iX=iX;  
    m_iY=iY;  
}  
void Point::SetX(int iX) {  
    m_iX=iX;  
}  
void Point::SetY(int iY) {  
    m_iY=iY;  
}  
int Point::GetX() {  
    return (m_iX);  
}  
int Point::GetY() {  
    return(m_iY);  
}
```

Aggregation - Example (2 of 3)

```
//Class Circle declaration
class Circle{
private:
    Point m_oCenter;
    int m_iRadius;
public:
    Circle(Point oP1,int iRadius);
    void SetCenter(Point oP1);
    void SetRadius(int iRadius);
    int GetRadius();
    float Area();
    Point& GetCenter();
};
```

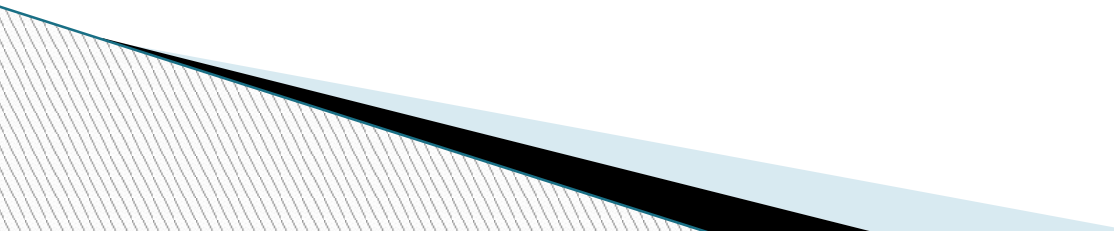
```
// Class Circle implementation
Circle::Circle(Point oP1,int iRadius) {
    m_oCenter=oP1;
    m_iRadius=iRadius;
}
void Circle::SetCenter(Point oP1) {
    m_oCenter=oP1;
}
void Circle::SetRadius(int iRadius) {
    m_iRadius=iRadius;
}
int Circle::GetRadius() {
    return(m_iRadius);
}
float Circle::Area() {

    return(3.14*m_iRadius*m_iRadius);
}
Point& GetCenter() {
    return(m_oCenter);
}
```

Aggregation - Example (3 of 3)

```
int main(int argc,char** argv) {  
    Point oCenter(100,100); // Point object representing  
    (100,100)  
    Circle oC1(oCenter,10); //Circle Object with center  
    (100,100) and radius 10  
  
    oC1.GetCenter().SetX(50);  
    oC1.GetCenter().SetY(50);  
    oC1.SetRadius(3);  
  
    printf("\nArea=%d",oC1.Area());  
  
    return 0;  
}
```

Operator Overloading - A feature of C++

- ▶ provides the idea of ***Extensibility***.
 - ▶ That is adding new definitions to the existing operators. In other words, giving extra work to the operator that already has some work.
 - ▶ However this is not commonly used and hence is not discussed here
- 

Input/Output Statements - cin and cout

- **cin** is used for accepting data from the user
- **cout** is used for displaying information on the monitor
- **cin** and **cout** are declared in the header file **iostream.h**, So, it should be included using **#include<iostream>**

- The syntax of **cout** (statement):

cout << String-Mesg/variable-name1 << String-Mesg/variable-name2 ... ;

- Here, << is called insertion operator which inserts output to Output Device (Monitor).
- Typically the data in the output stream may get onto the monitor by default or can be redirected to files or any other output device.

- **Examples:**

```
cout<<"Hello World" ;
```

```
cout<<"Infosys Technologies Ltd., \n";
```

```
cout<<"Employee Name : " << acName << endl;
```

```
cout<<"\t\t Employee Number : " << iEmpID << "\n";
```

Input/Output Statements - cin and cout

- ▶ The syntax of **cin** (statement):

cin >> variable-name1 >> variable-name2 >> variable-name3 ...;

- ▶ Here, >> is called extraction operator which extracts input from keyboard to variable.

- ▶ **Examples:**

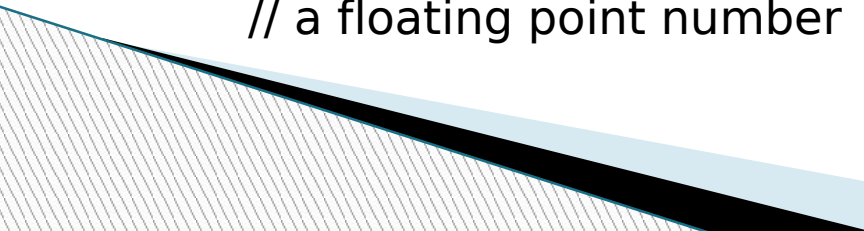
```
cout<< " Enter your name : ";
```

```
cin >> acName ;           // reads a string
```

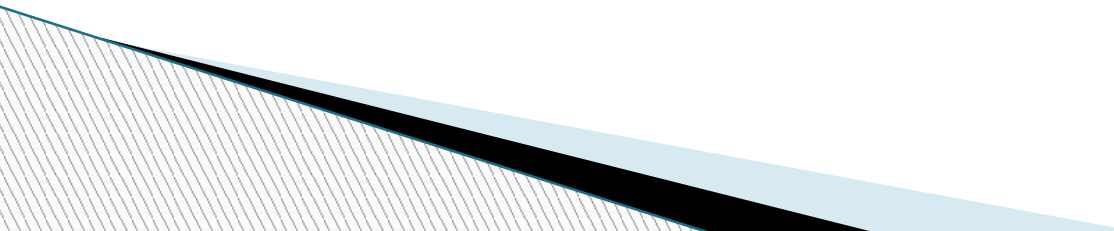
```
cout<< " Enter your Emp_ID and Salary : ";
```

```
cin >> iEmpID >> fSalary;  // reads an integer and
```

```
// a floating point number
```

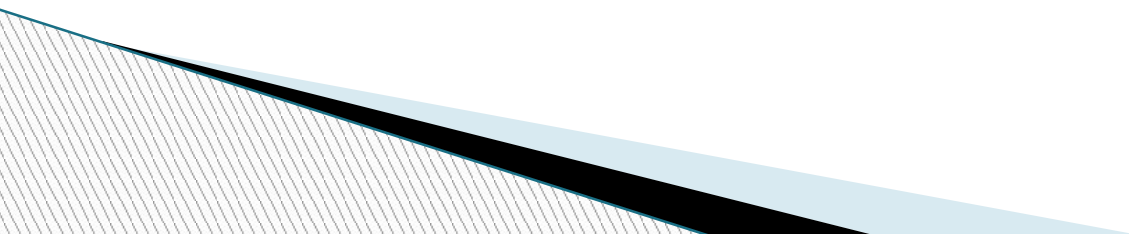


Summary

- ▶ Dynamic Memory Allocation
 - ▶ Inline Functions
 - ▶ Static Members
 - ▶ Const Members
 - ▶ Polymorphism
 - ▶ Function/Method Overloading
 - ▶ Ambiguity in Function Overloading
 - ▶ this pointer
 - ▶ Aggregation
 - ▶ Cin & Cout
- 

Session plan - Day 3

- ▶ Inheritance
- ▶ Method Overriding
- ▶ Base class pointers pointing to Derived class
- ▶ Virtual Functions and Dynamic Binding
- ▶ Abstract classes
- ▶ Different types of Inheritance



Inheritance

- ▶ The process of deriving a new class from an existing class is called as **Inheritance**
- ▶ The existing class is referred to as the **base class or super class** and the new class is called the **derived class** or **subclass**.
- ▶ Through Inheritance, C++ strongly supports the idea of **Reusability**. That is making use of the existing features to create the new feature.
- ▶ All the objects in this world come under some kind of classification. Inheritance allows the creation of **hierarchical classification**.
- ▶ For example, the bike “**Yamaha**” is a kind of ‘**Two Wheeler**’ which is again a kind of ‘**Vehicle**’.

Inheritance - Generalization & Specialization

- Generalization and Specialization are associated with the concept of Inheritance
- The Trainee class derived from Employee class
- Employee class is a generalization of Trainee class
- Trainee class is a specialization of the Employee class
- The relationship between Derived class and Base class is 'Is-A'



**Base
Class**



**Derived
Class**

Inheritance - Creating derived classes

- ▶ The general form of deriving the properties from one class to another class is :

```
class <derived-class-name> : [visibility-mode] <base-classname>
{
    // members of derived class

};
```

- Here, the **visibility mode** is optional and , if present, may be either private or public or protected.
- The default visibility mode is *private*.
- Visibility mode specifies whether the features of the base class are privately derived or publicly derived.

Inheritance - Example

```
class Employee {  
private:  
    int m_iEmpId;  
    float m_fBasic;  
    float m_fSal;  
public:  
    void CalculateSal() {  
  
        m_fSal=m_fBasic+.4*m_fBasic;  
    }  
};
```

```
Class Trainee : public Employee {  
private:  
    float m_fTPI;  
public:  
    void CalculateTPI(char cGrade) {  
        if(cGrade=='A') {  
            m_fTPI=1200;  
        }else {  
            m_fTPI=900;  
        }  
    }  
}
```

```
Employee oE1;  
Trainee oT1;
```

oE1.CalculateTPI();//Invalid

oE1.CalculateSal(); //Valid
oT1.CalculateSal(); //Valid
oT1.CalculateTPI(); //Valid

Inheritance - Memory allocation for objects

```
class Employee {  
private:  
    int m_iEmpId;  
    float m_fBasic;  
    float m_fSal;  
public:  
    void CalculateSal() {  
  
        m_fSal=m_fBasic+.4*m_fBasic;  
    }  
};
```

```
Class Trainee : public Employee {  
private:  
    float m_fTPI;  
public:  
    void CalculateTPI(char  
    cGrade) {  
        //Code to calculate TPI  
    }  
};
```

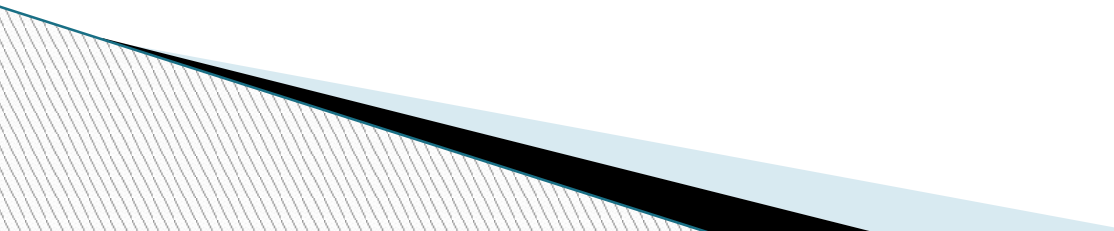
Employee oE1;

m_iEmpId
m_fBasic
M_fSal

Trainee oT1;

m_iEmpId
m_fBasic
m_fSal
m_fTPI

Inheritance - Protected Members

- ▶ The '**private**' members cannot be accessed outside the scope of the class.
 - ▶ The derived class also cannot access the private members.
 - ▶ Sometimes, we want the **derived class to access the private members** of the base class but at the same time, we don't want other classes or functions to access those members thru the objects.
 - ▶ Those members come under **protected** access specifier.
 - ▶ In other words, **protected members** are private to all, but public to its derived classes.
- 

Inheritance - Protected Members - Example

```
class Employee {  
private:  
    int m_iEmpId;  
protected:  
    float m_fBasic;  
    float m_fSal;  
public:  
    void CalculateSal() {  
  
        m_fSal=m_fBasic+.4*m_fBasic;  
    }  
};
```

```
Class Trainee : public Employee {  
private:  
    float m_fTPI;  
public:  
    void CalculateTPI(char cGrade)  
    {  
        if(cGrade=='A')  
            m_fTPI=1200;  
        }else {  
            m_fTPI=900;  
            m_fSal=m_fBasic+m_fTPI;  
            printf("%d",m_iEmpID,m_fSal);  
        }  
    }  
};
```

Valid

Invalid

```
Trainee oT1;  
Employee oE1;  
oE1.m_fBaisic // Invalid
```

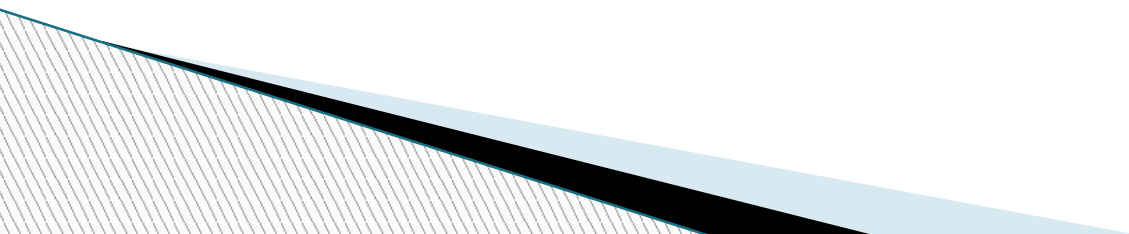
Inheritance - Protected Members (Continued)

Advantages:

- Derived classes can modify values of members of base directly
- Slight increase in performance; Avoids set/get function call overhead

Disadvantages:

- **No validity checking**
 - Derived class can assign illegal values
- **Implementation dependent**
 - Derived class member functions more likely dependent on base class implementation
 - Base class implementation changes may result in derived class modifications resulting in Fragile (brittle) software



Inheritance - Types of

▶ Inheritance

In C++, Inheritance can be classified based on two aspects namely,

- Based **on access Specifiers** while deriving the classes

public Inheritance

protected Inheritance

private Inheritance

- Based **on structure of classes**

Single Inheritance or Simple Inheritance

Multiple Inheritance (very rarely used)

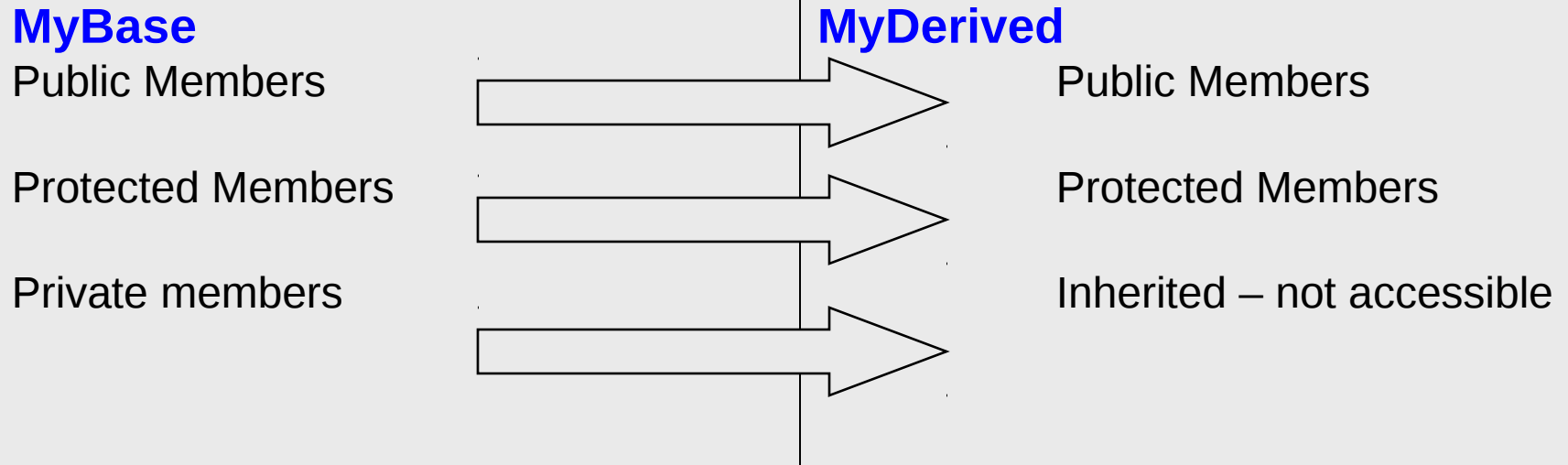
Multilevel Inheritance

Hierarchical Inheritance

Hybrid Inheritance (very rarely used)

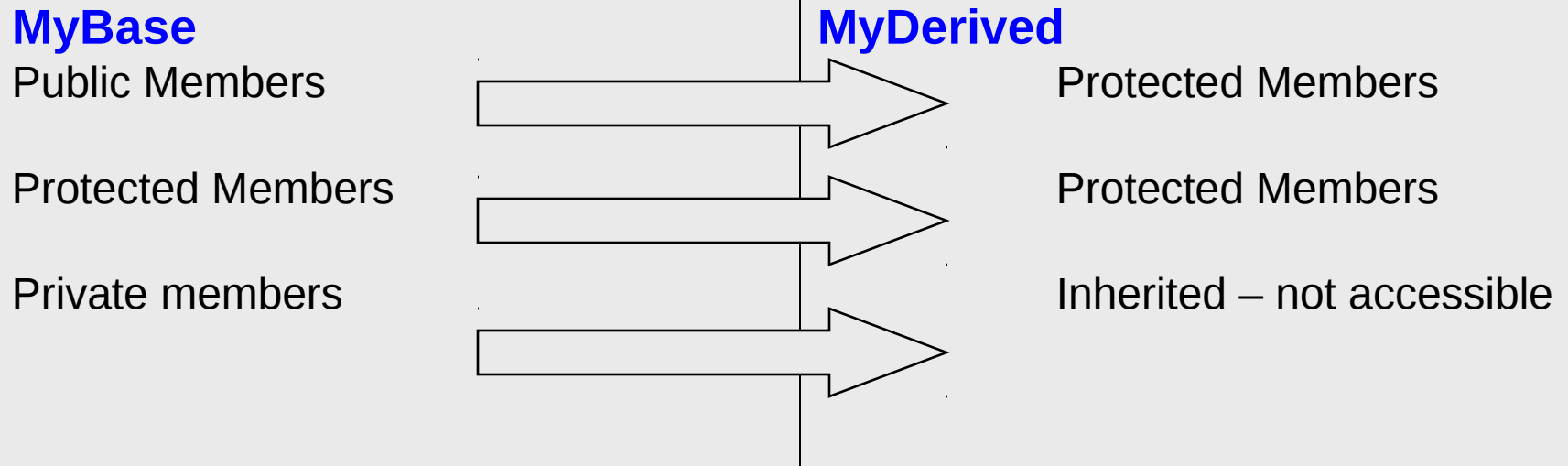
Inheritance - Public Inheritance

```
class MyDerived : public MyBase {  
    //Class MyDerived will inherit protected and public members of MyBase  
    //with no change in access specifier  
}
```



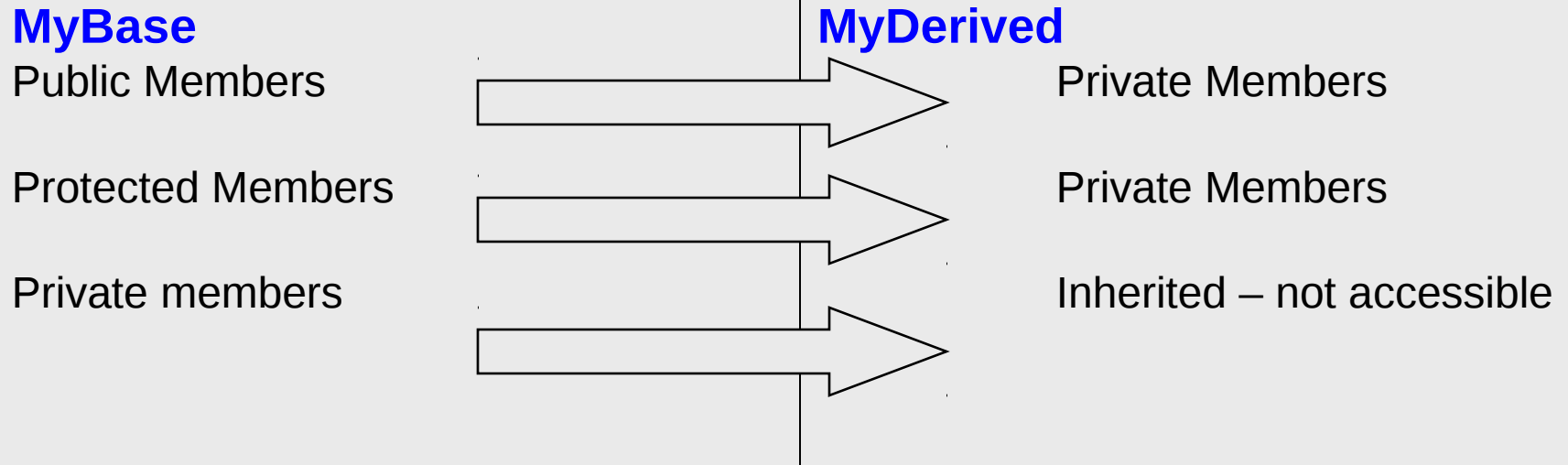
Inheritance - Protected Inheritance

```
class MyDerived : protected MyBase {  
    //Class MyDerived will inherit protected and public members of MyBase  
    //with public members getting promoted to protected members  
}
```



Inheritance - Private Inheritance

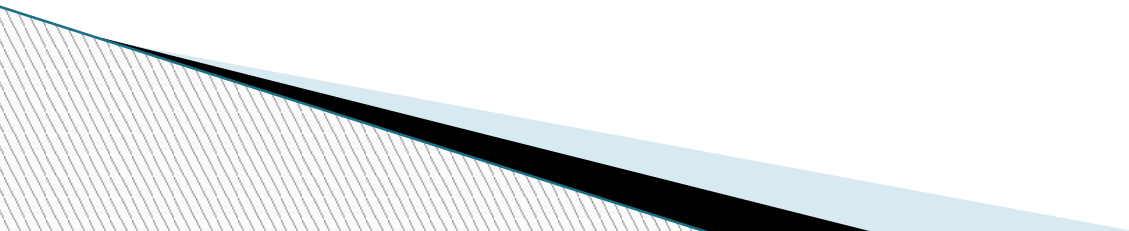
```
class MyDerived : private MyBase {  
    //Class MyDerived will inherit protected and public members of MyBase  
    //with protected and public members getting promoted to private  
}
```



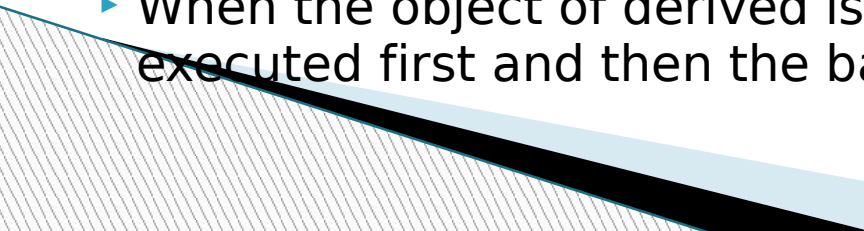
Inheritance - Summary

Type of Inheritance	Accessible inside Derived Class? i.e. In any of the methods of derived class			Accessible outside Derived Class? i.e. through derived class objects		
	Private member of Base class	Protected member of Base Class	Public member of Base class	Private member of Base class	Protected member of Base class	Public member of Base class
Private	NO	YES	YES	NO	NO	NO
Protected	NO	YES	YES	NO	NO	NO
Public	NO	YES	YES	NO	NO	YES

Inheritance Types - Based on structure



Inheritance - Constructors & Destructors

- ▶ Constructors & Destructors are not inherited whatever may be the access specifier used.
 - ▶ When you create object of Derived, Base class **default** constructor is executed first and then derived class constructor is executed
 - ▶ If you want to execute a specific Base class constructor during derived class object creation, you can specify that in derived class constructor header.
 - ▶ In case of multiple inheritance base class constructor is executed in the order they appear in derived class declaration.
 - ▶ In case of multilevel inheritance, constructors are executed in the order of inheritance.
 - ▶ When the object of derived is destroyed, destructor of derived is executed first and then the base class destructor is executed.
- 

Inheritance - Constructors & Destructor

- ▶ General syntax for calling base class constructor from derived class

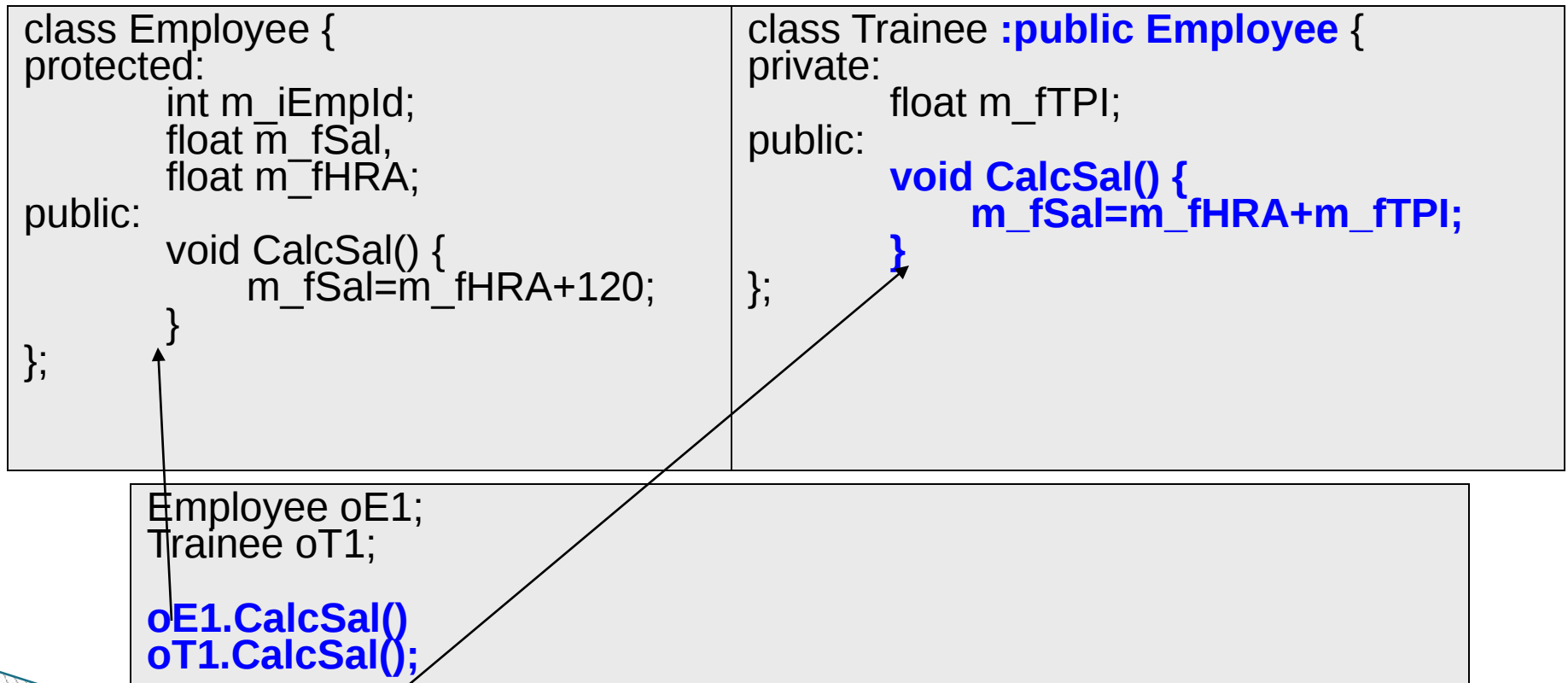
<Derived class Constructor >:<Base Class Constructor>(parameters) {}

```
class Employee {  
    private: int m_iEmpId; float m_fBasic, m_fSal;  
public:  
    Employee(int iEmp,float fBasic) {  
        // Base Constructor  
    }  
};
```

```
Class Trainee : public Employee {  
    private:  
        float m_fTPI;  
    public:  
        Trainee(int iEmp,float fBasic,float fTPI) : Employee(iEmp,fBasic) {  
            // Derived Constructor  
        }  
};
```


Method Overriding

- ▶ Redefining the methods in the derived class which is already defined in the base class is called method overriding.
- ▶ When this method is invoked by derived class object, the method defined in the derived class gets executed.



Base class pointer & Derived class objects

```
class Employee {  
protected:  
    int m_iEmpId;  
    float m_fSal,m_fHRA;  
public:  
    void CalcSal() {  
        m_fSal=m_fHRA+120;  
    }  
}
```

```
class Trainee :public Employee {  
private:  
    float m_fTPI;  
public:  
    void CalcSal() {  
        m_fSal=m_fHRA+m_fTPI;  
    }  
}
```

```
Employee* poE1;  
poE1= new Employee();  
poE1->CalcSal();
```

```
Employee* poE1;  
poE1= new Trainee();  
poE1->CalcSal();  
  
/*base class pointer type casted*/  
(Trainee*)poE1->CalcSal();
```

Virtual Functions - Dynamic Polymorphism

- ▶ It is also possible to make a base class pointer to invoke a method in the derived class without type casting
- ▶ This is done using the concept of virtual functions.
- ▶ The following declares a virtual function:

```
virtual void CalcSal();
```

```
Employee * poE1 = new Trainee();  
poE1->CalcSal();
```

- ▶ This invokes the **Trainee::CalcSal()** using base class pointer.
- ▶ Here binding function call with the function definition happens during program execution. This is called as ***Dynamic Binding***.

Virtual Functions - Dynamic Polymorphism -

```
class Employee {  
protected:  
    int m_iEmpId;  
    float m_fSal;  
    float m_fHRA;  
public:  
    virtual void CalcSal() {  
        m_fSal=m_fHRA+120;  
    }  
};
```



```
Employee* poE1;  
poE1= new Employee();  
poE1->CalcSal();
```

```
class Trainee :public Employee {  
private:  
    float m_fTPI;  
public:  
    void CalcSal() {  
        m_fSal=m_fHRA+m_fTPI;  
    }  
};
```



```
Employee* poE1;  
poE1= new Trainee();  
poE1->CalcSal();
```

Inheritance - Abstract Class

Pure Virtual Functions

- A **pure virtual function** is a virtual function that has **no definition** within the base class.
- If a class contains at least one **pure virtual function**, that class is called as **abstract class**.
- The derived class ***should*** implement the pure virtual function. If not, derived class also will be an abstract class.
- **Abstract class CANNOT BE instantiated.** i.e. object of abstract class cannot be created.
- The only purpose of abstract class is **Generalization**. i.e. It declares the prototype of collection of similar classes in the hierarchy of classes (**Standard interface**).
- A general form of declaring pure virtual function is :
virtual <return-type> <func-name> ([parameter-list]) = 0;
- Pointer to an abstract class can be declared

Abstract Class - Example

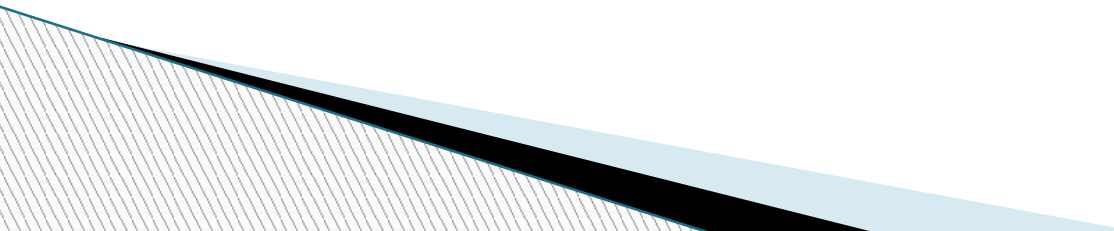
```
class Employee {  
protected:  
    int m_iEmpId;  
    float m_fSal;  
    float m_fHRA;  
public:  
    virtual void CalcSal() =0;  
};
```

```
class Trainee :public Employee {  
private:  
    float m_fTPI;  
public:  
    void CalcSal() {  
        m_fSal=m_fHRA+m_fTPI;  
    }  
};
```

```
Employee oE1; // Invalid  
Employee* poE1; //Valid  
poE1= new Employee();// Invalid
```

```
Employee* poE1;  
poE1= new Trainee(); //Valid  
poE1->CalcSal(); //Valid
```

Summary

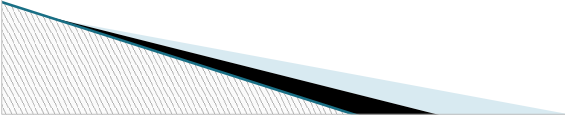
- ▶ Method Overriding
 - ▶ Pointer to objects
 - ▶ Base class pointers pointing to Derived class
 - ▶ Virtual Functions and Dynamic Binding
 - ▶ Abstract classes
 - ▶ Different types of Inheritance
- 



C++ Programming

Dr.Mohan.S

Session Plan

- Programming Techniques
 - Introduction to Object Oriented Concepts
 - Basics of C++
 - Reference Variables
 - Parameter Passing techniques
 - Abstract Data types
 - Classes and Objects
 - Access Specifiers
 - Class – Best Practices and Guidelines
 - Constructors and Destructors
- 



Operators Common in C and C++

- Arithmetic operators `+, -, *, /, %`
- Pre and Post Unary Operators `++, --`
- Assignment operator `=`
- Compound Assignment Operators `=, +=, -=, /=, *=, %=`
- Relational Operators `>, >=, <, <=, ==, !=`
- Logical Operators `!, &&, ||`
- Address Operator `&`
- **sizeof** operator
- Bitwise logical operators (`&` - AND, `|` - OR, `!` - XOR, `~` - NOT)
- Bitwise Shift operators (`<<` - Left Shift, `>>` - Right Shift)
- Conditional operator (`? :`) also called as Ternary Operator
 - Syntax:
 - `<var-name> = (<condition>) ? value-if-true : value-if-false ;`
 - `iMax=(iNum1>iNum2) ? iNum1 : iNum2 ;`

The basic arithmetic operators help us in performing the arithmetic operations. The `%` (modulo operator) is used to find the remainder of a divide operation

eg. `int x=5,y=2;`

`int z=5/2;` will store 2 in z

`int z=5%2` will store 1 in z

`%` operator cannot be applied to float data type.

Hence the instruction `5.2 % 2.5` will result in error

For comparing if 2 values are equal or not remember to use `==` and not a single `=`. `==` indicates comparisons and `=` indicates assignment operation

eg. `int x=5,y=3`

`x==y` will be considered as an conditional expression that will be evaluated to true(non-zero) for false(zero)

`x=y` will be considered as an assignment expression which will assign 3 (value of y) to x. x after the instruction will have value 3.

Operators in C++

- Scope Resolution Operator (`::`)
- Memory Allocation/De-allocation operator (**new** and **delete**)
- Member Access operators (`->` , `.*` , `->*` , `..`)
- Reference Operator (**&**)
- Insertion and Extraction Operators (**<<** **and** **>>**).

Logical operators available are ! (NOT), && (AND), || (OR)

eg. consider 3 variables x,y,z containing values 1,2,3 respectively

x=1, y=2, z=3

the condition `x==1 && y==3` will evaluate to false

the condition `x==1 && z==3` will evaluate to true

the condition `x==1 || y==3` will evaluate to true

the condition `x==1 || z==3` will evaluate to true

the condition `!(x < 2 && y==2)` will evaluate to false

AND table followed for evaluation is : T && T~T, T && F~F, F && T~F, F && F~T

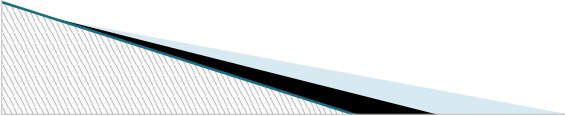
OR table followed for evaluation is : T || T~T, T || F~T, F || T~T, F || F~F

NOT table followed is : ! T~F, ! F~T

Classification of Operators:

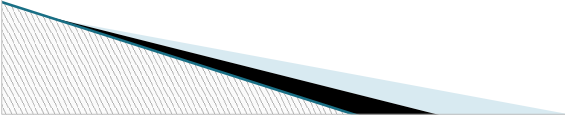
(1) Based on the number of operands:

Summary

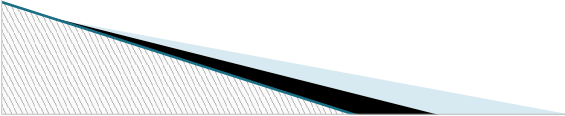
- ▶ Programming Techniques
 - ▶ Introduction to Object Oriented Concepts
 - ▶ Basics of C++
 - ▶ Reference Variables
 - ▶ Parameter Passing techniques
 - ▶ Abstract Data types
 - ▶ Classes and Objects
 - ▶ Access Specifiers
 - ▶ Class – Best Practices and Guidelines
 - ▶ Constructors and Destructors
- 



Session Plan 2

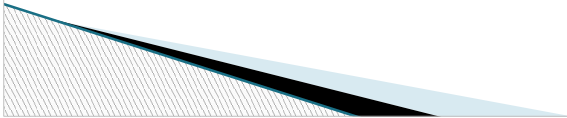
- ▶ Dynamic Memory Allocation
 - ▶ Inline Functions
 - ▶ Static Members
 - ▶ Const Members
 - ▶ Polymorphism
 - ▶ 'this' pointer
 - ▶ Aggregation
 - ▶ Cin & Cout
- 

Summary

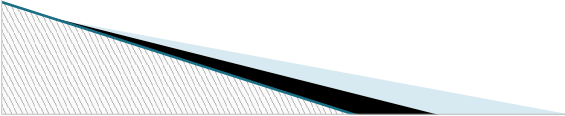
- ▶ Dynamic Memory Allocation
 - ▶ Inline Functions
 - ▶ Static Members
 - ▶ Const Members
 - ▶ Polymorphism
 - ▶ Function/Method Overloading
 - ▶ Ambiguity in Function Overloading
 - ▶ this pointer
 - ▶ Aggregation
 - ▶ Cin & Cout
- 

Session plan - Day 3

- ▶ Inheritance
- ▶ Method Overriding
- ▶ Base class pointers pointing to Derived class
- ▶ Virtual Functions and Dynamic Binding
- ▶ Abstract classes
- ▶ Different types of Inheritance



Summary

- ▶ Method Overriding
 - ▶ Pointer to objects
 - ▶ Base class pointers pointing to Derived class
 - ▶ Virtual Functions and Dynamic Binding
 - ▶ Abstract classes
 - ▶ Different types of Inheritance
- 

How to debug a program for logical errors
Refer to GDB slides for SC
For LC it is already covered in C
programming.